The Search for Optimal Sparse Decision Trees

The Challenge of Optimality

- Decision trees remain highly popular due to their inherent interpretability, crucial for high-stakes domains (medicine, criminal justice).
- However, traditional algorithms (CART, C4.5) rely on greedy top-down splitting and post-hoc pruning. They provide **no guarantee of optimality**.
- The search space of all possible binary trees is astronomically large (e.g., $O(10^{28})$ for depth 4 with 20 features).

Limitations of Alternatives

- **Mathematical Programming:** Existing Mixed-Integer formulations (e.g., OCT, BinOCT) struggle to scale or arbitrarily constrain tree topologies.
- **Neural Networks:** Provide high accuracy and scalability but sacrifice transparency entirely.
- **Our Goal:** The first practical algorithm to search the unconstrained tree space and return a *provably optimal* decision tree.

Optimization Framework: Branch-and-Bound

The objective is to minimize regularized empirical risk (misclassification + sparsity):

Conceptual Formulation

$$\min_{\text{tree}} \hat{L}(\text{tree}) = (1/n) \sum 1_{\{\text{tree}(x_i) \neq y_i\}} + C(\#\text{ leaves})$$

OSDT Implementation

$$\min_{d} R(d, x, y) = l(d, x, y) + \lambda H_d$$

Where d is the tree, the sum or l is the misclassification error, H_d is the number of leaves, and λ is the penalty.

1. Leaf-Based Representation

A tree is represented as a set of leaves: $d = (d_{un}, d_{split})$.

d_{un}: Fixed leaves (will not be split further).

d_{split}: Leaves available for splitting.

This isolates the fixed portion of the tree for bound calculations.

2. The Optimization Loop

Maintains a priority queue of partial trees.

Pop: Extract tree d from queue.

Bound: If lower bound $b \geq R^c$ (current best), prune the tree and all its descendants.

Branch: Generate child trees by splitting d_{split}.

Analytical Bounds for Search Space Pruning

To make the search tractable, we establish theorems to aggressively prune the search space:

Hierarchical Objective Lower Bound

The loss decomposes into fixed leaves and split leaves. By assuming zero error for d_{split}, we get a strict lower bound: $b(d_{un}) = l_p(d_{un}) + \lambda H_d \leq R(d', x, y)$. Any descendant tree d' cannot have an objective lower than $b(d_{un})$.

Equivalent Points Bound

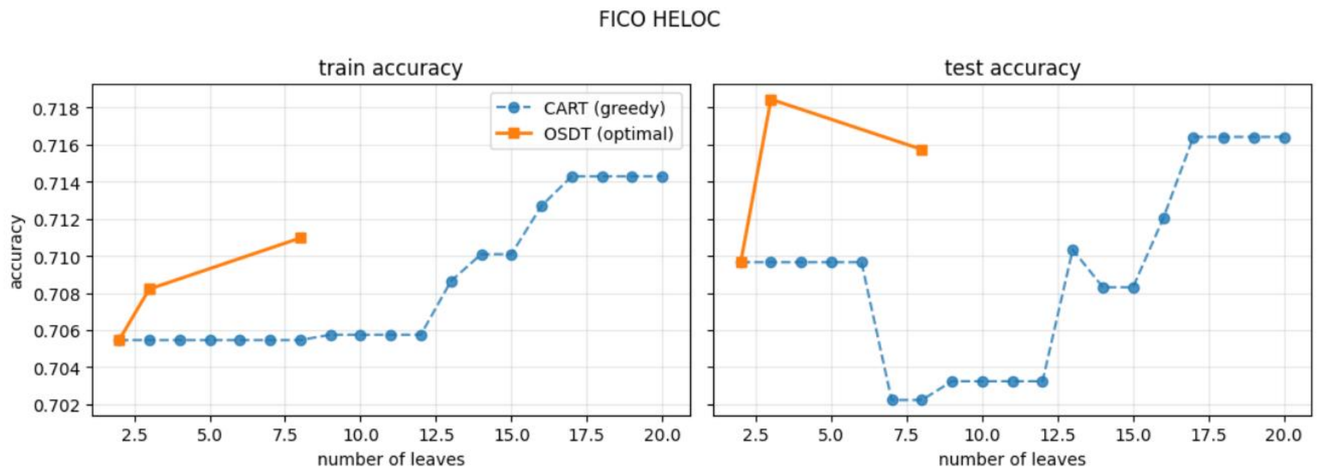
Data samples with identical features but different labels represent inevitable classification errors. Adding the minority label count of these groups to $b(d_{un})$ tightens the lower bound significantly.

Lower Bound on Node Support & Accuracy

Support: The data fraction traversing an internal node must be ≥ 2λ.
Accuracy: The incremental accuracy gained by a split must be ≥ λ, otherwise the branch must be further split or pruned entirely.

Experimental Results: FICO HELOC Dataset

Sparsity vs. Accuracy (OSDT vs. CART)



The Greedy Penalty: Under 5-fold cross-validation (λ = 0.002), OSDT restricted to 3 leaves achieves a mean test accuracy of **71.13%**. CART, constrained to the exact same 3-leaf budget, achieves only **70.67%**.

Global Optimality: CART's top-down greedy splitting permanently traps it in suboptimal configurations. OSDT's global optimization mathematically guarantees the most accurate possible tree for the chosen sparsity level.

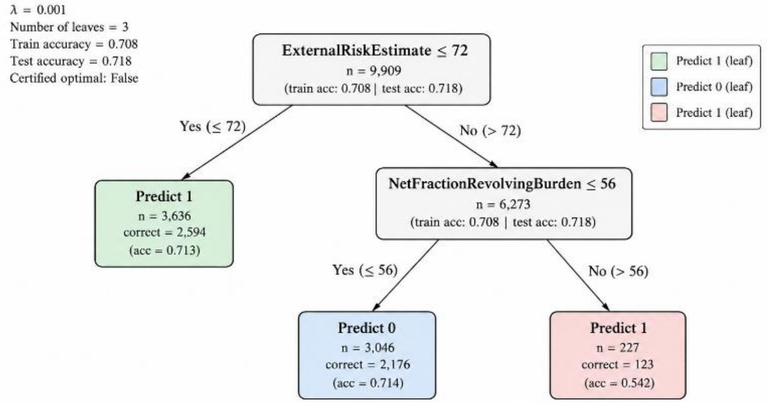


Figure 1: Interpretable decision tree for FICO HELOC (λ = 0.001). Each node shows the splitting rule (if applicable), the number of samples (*n*), the number of correct predictions, and the node accuracy (in parentheses).

Interpretable Output: At λ = 0.001, OSDT produces this highly interpretable 3-leaf model.

Stump Optimality: If λ is increased to ≥ 0.005, the penalty for adding leaves outweighs the ≈ 0.001 accuracy gain per leaf, and OSDT mathematically certifies a 2-leaf stump as the global optimum.

Conclusion and Future Work

Summary of Contributions

- **Provable Optimality:** The first algorithm to efficiently explore the unconstrained binary tree space and balance accuracy with sparsity.
- **Algorithmic Innovation:** Co-design of theoretical pruning bounds and priority-queue scheduling (Branch-and-Bound framework).
- **Systems Engineering:** Over 97% reduction in execution time achieved through permutation caching, LeafCache representations, and fast bit-vector operations.

Open Challenges & Next Steps

- **Continuous Variables:** Current implementations rely heavily on pre-binarized features. Direct optimization over continuous splits remains non-trivial.
- **Objective Generalization:** Expanding the formulation beyond 0-1 misclassification loss to support ranking, regression, and survival analysis objectives.
- **Scalability Limits:** While significantly faster than prior work, scaling to ultra-high dimensional feature spaces requires further bounding innovations.